DIGITAL LOGIC AND COMPUTER DESIGN
UNIT 1 — DESCRIPTIVE QUESTIONS WITH ANSWERS

PES University | Descriptive Question Bank | T1: Harris & Harris, 2nd Edition

1. What is meant by a divide-by-3 circuit?

Model Answer / Explanation: A divide-by-3 circuit produces an output signal whose frequency is one-third of the input clock frequency. Therefore, f_out = f_in / 3.

2. How many states are required in a basic divide-by-3 FSM?

Model Answer / Explanation: Three states are required: S0, S1, and S2. The FSM repeatedly follows S0 → S1 → S2 → S0.

3. What is the next state after S2 in a divide-by-3 FSM?

Model Answer / Explanation: The next state is S0. Returning to S0 completes one cycle of the three-state sequence and starts the next cycle.

4. What is the purpose of a reset signal in the divide-by-3 FSM?

Model Answer / Explanation: Reset initializes the FSM to a known state, usually S0. This ensures predictable operation when the circuit starts.

5. What is the output frequency when a 30 MHz clock is divided by 3?

Model Answer / Explanation: The output frequency is 10 MHz because f_out = 30 MHz / 3 = 10 MHz.

6. Draw or describe the state sequence for a Moore FSM that detects 1011.

Model Answer / Explanation: Use five states: S0: no bits matched; S1: matched 1; S2: matched 10; S3: matched 101; S4: matched 1011. The main path is S0 --1--> S1 --0--> S2 --1--> S3 --1--> S4. In S4, output = 1. For overlap, S4 transitions to S1 on input 1 and to S2 on input 0.

7. Explain the difference between Moore and Mealy pattern recognizers.

Model Answer / Explanation: In a Moore FSM, output depends only on the present state, so a separate detection state is usually required. In a Mealy FSM, output depends on both present state and input, so detection can occur on a transition. A Moore design is often easier to interpret and has more stable state-based outputs, while a Mealy design may use fewer states and respond one transition earlier.

8. Write the state encoding for a Moore recognizer of 1011 and explain it.

Model Answer / Explanation: One possible encoding is S0=3'b000, S1=3'b001, S2=3'b010, S3=3'b011, and S4=3'b100. Three bits are required because five states must be represented, and 2^2=4 is insufficient while 2^3=8 is sufficient.

9. Explain the role of the state register and next-state logic in Verilog.

Model Answer / Explanation: The state register stores the present state and is updated on the active clock edge. The next-state combinational block examines the present state and input, then calculates the state to be loaded on the next clock edge. This separation makes the FSM easier to design and verify.

10. Explain why a default case is included in the Verilog state-transition block.

Model Answer / Explanation: A default case handles invalid or unused state encodings. Assigning next_state = S0 provides a safe recovery path and helps the FSM return to a known operating state if an unexpected state occurs.

11. Design a Moore FSM to detect the pattern 1011 with overlapping detection. Explain the states and transitions.

Model Answer / Explanation: States: S0=no match, S1=matched 1, S2=matched 10, S3=matched 101, S4=matched 1011. Transitions: S0: 0→S0, 1→S1. S1: 0→S2, 1→S1. S2: 0→S0, 1→S3. S3: 0→S2, 1→S4. S4: 0→S2, 1→S1. Output is 1 only in S4; otherwise it is 0. The transitions from S4 preserve overlap because the suffixes of 1011 may be prefixes of a new pattern.

12. Write Verilog code for a Moore FSM that recognizes 1011 with overlapping detection. Explain the major blocks.

Model Answer / Explanation:
```
module moore_1011(
    input wire clk, reset, x,
    output reg detect
);
localparam S0=3'b000, S1=3'b001,
           S2=3'b010, S3=3'b011,
           S4=3'b100;
reg [2:0] state, next_state;

always @(posedge clk or posedge reset)
    if (reset) state <= S0;
    else       state <= next_state;

always @(*) begin
    case (state)
      S0: if (x) next_state=S1; else next_state=S0;
      S1: if (x) next_state=S1; else next_state=S2;
```

```
      S2: if (x) next_state=S3; else next_state=S0;
      S3: if (x) next_state=S4; else next_state=S2;
      S4: if (x) next_state=S1; else next_state=S2;
      default: next_state=S0;
    endcase
end

always @(*)
    if (state==S4) detect=1'b1;
    else           detect=1'b0;
endmodule
```

Explanation: The first block stores the present state. The second block implements the transition table. The third block is Moore output logic, so detect depends only on state and is high only in S4.

13. Develop the state table for a Moore 1011 pattern recognizer and explain the output.

Model Answer / Explanation: State table: S0: x=0→S0, x=1→S1, output=0. S1: x=0→S2, x=1→S1, output=0. S2: x=0→S0, x=1→S3, output=0. S3: x=0→S2, x=1→S4, output=0. S4: x=0→S2, x=1→S1, output=1. Since this is a Moore machine, the output is associated with the present state rather than directly with the input.

14. Explain how overlapping detection is implemented in a Moore FSM for 1011.

Model Answer / Explanation: After reaching S4, the FSM does not always return to S0. If the next input is 1, it goes to S1 because the most recent 1 can begin a new pattern. If the next input is 0, it goes to S2 because the ending sequence 10 matches the first two bits of the target pattern. These transitions retain useful partial matches and allow overlapping occurrences to be detected.

15. Compare the hardware and timing characteristics of Moore and Mealy pattern recognizers.

Model Answer / Explanation: A Moore recognizer generally uses a separate output state, so it may require more states than an equivalent Mealy recognizer. Its output depends only on registered state and is therefore state-synchronous and less directly sensitive to input changes. A Mealy recognizer can assert output on the transition that receives the final pattern bit, often using fewer states and responding earlier. However, because its output depends directly on input and state, careful timing design is needed to avoid unwanted output changes.

16. Differentiate Half Adder and Full Adder with truth tables.

Detailed Answer:
Half Adder:
A Half Adder adds two 1-bit binary numbers. It has two inputs (A,B) and two outputs (Sum and Carry).
Boolean equations:
Sum = A XOR B
Carry = A AND B

Truth Table:
A B | S C
0 0 | 0 0
0 1 | 1 0
1 0 | 1 0
1 1 | 0 1

Full Adder:
A Full Adder adds three binary inputs (A,B,Cin). It has outputs Sum and Carry.
Sum = A XOR B XOR Cin
Carry = AB + Cin(A XOR B)

A Full Adder can be implemented using two Half Adders and one OR gate. Unlike a Half Adder, it can accept an input carry, making it suitable for multi-bit addition.

17. Explain the working of a 4-bit Ripple Carry Adder with its advantages and disadvantages.

Detailed Answer:
A 4-bit Ripple Carry Adder is constructed by connecting four Full Adders in cascade.
The carry output from each Full Adder becomes the carry input to the next stage.

Working:
1. The least significant Full Adder receives the external carry input.
2. Each Full Adder computes Sum and Carry.
3. The carry propagates from LSB to MSB until the final carry is produced.

Advantages:
• Simple design.
• Requires less hardware.

Disadvantages:
• Large propagation delay because every stage waits for the previous carry.
• Not suitable for high-speed processors.

18. Explain Carry Lookahead Adder using Generate and Propagate signals.

Detailed Answer:
Carry Lookahead Adders reduce delay by computing carries in parallel.

Generate (G) = A·B
Propagate (P) = A XOR B

Carry equations:
C1 = G0 + P0C0
C2 = G1 + P1G0 + P1P0C0

Because carries are calculated simultaneously rather than waiting for previous stages, the CLA is much faster than the Ripple Carry Adder. The trade-off is increased hardware complexity.

19. Explain binary subtraction using the Two's Complement method with an example.

Detailed Answer:
To subtract B from A:
1. Find the 2's complement of B.
2. Add it to A.
3. If a carry is generated, discard it.

Example:
A = 1011
B = 0110
1's complement of B = 1001
2's complement = 1010
1011 + 1010 = 10101
Discard carry = 0101

Therefore, 1011 - 0110 = 0101.

20. Explain the functions of an Arithmetic Logic Unit (ALU).

Detailed Answer:
The ALU is the computational unit of a processor.

Arithmetic operations:
• Addition
• Subtraction
• Increment

• Decrement

Logic operations:
• AND
• OR
• XOR
• NOT

Other operations:
• Shift left/right
• Compare
• Set-on-less-than

The Control Unit selects the required ALU operation using control signals.

21. Explain the operation of a binary comparator.

Detailed Answer:
A binary comparator compares two binary numbers.

Outputs:
A>B
A<B
A=B

Comparison starts from the most significant bit.
If MSBs are equal, the next lower bits are compared until a difference is found.

Applications:
• Sorting
• Address comparison
• Processor decision making
• Digital control systems.

22. Differentiate logical shift and arithmetic shift with examples.

Detailed Answer:
Logical Shift:
• Vacated bits are filled with zero.
• Used for unsigned numbers.

Example:
10110010 → Logical Right Shift

01011001

Arithmetic Shift:
• Sign bit is preserved.
• Used for signed numbers.

Example:
11010010 → Arithmetic Right Shift
11101001

Arithmetic shifts preserve the sign of signed integers.

23. Explain binary multiplication using the shift-and-add method.

Detailed Answer:
Binary multiplication is similar to decimal multiplication.

Example:
1101 × 0011

Partial products:
1101
11010

Adding partial products:
100111

Decimal verification:
13 × 3 = 39
39 = 100111_2

Shift-and-add multiplication is implemented in hardware multipliers.

24. Explain propagation delay in adders and its effect on performance.

Detailed Answer:
Propagation delay is the time required for a signal to travel from input to output.

In Ripple Carry Adders, every Full Adder must wait for the previous carry.
Hence,
Delay = Number of Full Adders × Delay per Full Adder.

Large delays reduce processor speed.
Carry Lookahead Adders minimize this delay by generating carries simultaneously.

25. Explain why multiplexers are used in ALUs.

Detailed Answer:
An ALU performs many operations simultaneously inside separate hardware blocks.
A multiplexer selects one output according to control signals.

Advantages:
• Reduces hardware duplication.
• Enables multiple operations using one output path.
• Improves processor flexibility.

For example, if control signal = 00, ADD output is selected; 01 selects SUB; 10 selects AND; 11 selects OR.

26. Represent the decimal number 9.25 in fixed-point binary notation using 4 integer bits and 4 fraction bits.

Solution:

- Integer part: 9 = 1001_2
- Fraction part: 0.25 = 2^{-2} → bit pattern 0100
- Combine: 1001.0100

Final Answer: 1001.0100

27. Convert the fixed-point binary number 0011.1010 (4 integer bits, 4 fraction bits) to its decimal value.

Solution:

- Integer part: 0011 = 3
- Fraction part: 1010 → $2^{-1} + 2^{-3} = 0.5 + 0.125 = 0.625$
- Total = 3 + 0.625 = 3.625

Final Answer: 3.625

28. Encode the decimal number 350 as a 32-bit IEEE 754 single-precision floating-point number.

Solution:

1. Convert 350 to binary: $350 = 101011110_2$
2. Normalize: $101011110 = 1.01011110 \times 2^8$
3. Sign bit = 0 (positive)
4. Biased exponent = 8 + 127 = 135 = 10000111
5. Fraction (23 bits, drop implicit leading 1, pad with zeros): 01011110000000000000000

Final Answer: 0 10000111 01011110000000000000000

29. Encode the decimal number −0.15625 as a 32-bit IEEE 754 single-precision floating-point number.

Solution:

1. Convert 0.15625 to binary: $0.15625 = 2^{-3} + 2^{-5} = 0.00101_2$
2. Normalize: $0.00101 = 1.01 \times 2^{-3}$
3. Sign bit = 1 (negative)
4. Biased exponent = −3 + 127 = 124 = 01111100
5. Fraction (23 bits): 01000000000000000000000

Final Answer: 1 01111100 01000000000000000000000

30. Using the 8-step floating-point addition procedure, compute 12.5 + 0.375 and give the final IEEE 754 single-precision encoding of the result.

Solution: Step 1–2 (extract fields, prepend implicit 1):

- $12.5 = 1100.1_2 = 1.1001 \times 2^3$ (exponent = 3)
- $0.375 = 0.011_2 = 1.1 \times 2^{-2}$ (exponent = −2)

Step 3 (compare exponents):

- Difference = 3 − (−2) = 5

Step 4 (shift smaller mantissa):

- Express 0.375's mantissa relative to exponent 3: $1.1 \times 2^{-5} = 0.000011$ (shift right 5

bits)

Step 5 (add mantissas):

1.100100

+ 0.000011

1.100111

Step 6 (normalize):

- 1.100111 is already between 1 and 2 → no renormalization needed. Exponent stays 3.

Step 7 (round):

- Result is exact → no rounding necessary.

Step 8 (assemble):

- Sign = 0
- Biased exponent = 3 + 127 = 130 = 10000010
- Fraction = 100111 padded to 23 bits: 10011100000000000000000

Verification: $1.100111_2 \times 2^3 = 1100.111_2 = 12 + 0.5 + 0.25 + 0.125 = 12.875 = 12.5 + 0.375$

Final Answer: 12.875 → 0 10000010 10011100000000000000000